

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Вычислительной техники

ОТЧЕТ
по лабораторной работе № 7
по дисциплине «Алгоритмы и структуры данных»
Тема: КОМБИНИРОВАННЫЕ СТРУКТУРЫ ДАННЫХ
И СТАНДАРТНАЯ БИБЛИОТЕКА ШАБЛОНОВ

Студенты гр. 4312

Преподаватель

Устинов В. А.

Гущин К. Д.

Аббас С. А. М.

Санкт-Петербург

2025

Оглавление

1	Цель работы	3
2	Задание 36 на работу с деревьями	3
3	Контейнер BitMaskSet.....	3
4	Контейнер Sequence<T>	4
5	Контейнер HashTable<K, V>	4
6	Класс Generator	5
7	Оценка временной сложности Хеш-таблицы.....	5
8	Тестовые примеры операций	6
9	Результаты тестирования.....	8
10	Вывод	8

1 Цель работы

Изучение комбинированных структур данных и их применение для поддержки операций как над множеством, так и над последовательностью с использованием алгоритмов из стандартной библиотеки шаблонов (STL).

2 Задание 36 на работу с деревьями

36	XT	CONCAT, ERASE, EXCL
----	----	---------------------

Необходимо реализовать пользовательский контейнер на базе хеш-таблицы, поддерживающий как операции над множеством, так и над последовательностью. Для последовательностей реализовать три операции: CONCAT (сцепление двух последовательностей), ERASE (удаление диапазона элементов по индексам) и EXCL (исключение всех вхождений заданной подпоследовательности).

3 Контейнер BitMaskSet

Операция	Метод / оператор	Назначение
Объединение	operator	Объединяет два множества ($A \cup B$)
Пересечение	operator&	Пересечение множеств ($A \cap B$)
Разность	operator-	Элементы A, не принадлежащие B ($A \setminus B$)
Симметрическая разность	operator^	Элементы, принадлежащие ровно одному множеству ($A \oplus B$)
Дополнение	operator~	Дополнение до универсума
Присваивание с операцией	operator =, &=, ^=, -=	Совмещённые операции
Сравнение	operator==	Проверка равенства множеств
Вставка элемента	insert(char)	Добавляет элемент в множество
Удаление элемента	remove(char)	Удаляет элемент из множества
Проверка наличия	contains(char)	Проверяет, есть ли элемент
Очистка	clear()	Удаляет все элементы
Преобразование в строку	toString()	Вывод множества в читаемом виде
Формула варианта 36	variant36() (static)	Вычисляет $((A \oplus (B \cap C)) \setminus D) \cap E$

4 Контейнер Sequence<T>

Операция	Метод	Назначение
Конструктор	Sequence(std::initializer_list<T>)	Создание последовательности из списка
Доступ по индексу	at(size_t pos)	Доступ к элементу (индексация с 1)
Получение размера	size()	Возвращает количество элементов
Проверка на пустоту	empty()	Проверяет, пуста ли последовательность
Добавление в конец	push_back(const T&)	Добавляет элемент в конец
Очистка	clear()	Удаляет все элементы
Слияние	concat(const Sequence&)	Присоединяет другую последовательность
Удаление диапазона	erase(p1, p2)	Удаляет элементы с p1 по p2
Исключение подпоследовательности	excl(const Sequence&)	Удаляет первое вхождение подпоследовательности
Вывод	print()	Печатает последовательность в формате [a, b, c]
Сравнение	operator==, !=	Сравнивает две последовательности

5 Контейнер HashTable<K, V>

Операция	Метод	Назначение
Конструктор	HashTable(size_t)	Создаёт хеш-таблицу с заданным числом корзин
Хеширование	getBucketIndex(key)	Вычисляет индекс корзины для ключа
Вставка	insert(key, value)	Добавляет пару ключ-значение
Доступ	operator[](key)	Возвращает значение по ключу (создаёт при отсутствии)
Получение значения	getElem(key)	Возвращает значение по ключу (бросает исключение)

Проверка наличия	contains(key)	Проверяет, есть ли ключ в таблице
Удаление	erase(key)	Удаляет пару по ключу
Итератор	begin() / end()	Обход всех элементов таблицы
Вывод всех значений	printAllValues()	Отладочная печать таблицы

6 Класс Generator

Операция	Метод	Назначение
Конструктор	Generator()	Инициализация генератора случайных чисел
Генерация множества	generateSet(min, max)	Генерирует случайное множество заданного размера
Случайное число	randomInt(min, max)	Генерирует целое число в диапазоне

7 Оценка временной сложности Хеш-таблицы

Операция	Лучший случай	Средний случай	Худший случай
Вставка (insert, operator[])	O(1)	O(1)	O(1)
Поиск (contains, getElem)	O(1)	O(1)	O(n)
Удаление (erase)	O(1)	O(1)	O(n)
Доступ по ключу (operator[])	O(1)	O(1)	O(n)

8 Тестовые примеры операций

```
=====
Variant 36 | Power: 32 | (A ⊕ B ∩ C) \ D ∩ E
Priority: ∩ > ⊕ > \ > U
Formula expands as: ((A ⊕ (B ∩ C)) \ D) ∩ E
=====

=== Example 1: Manual Sets ===
A = {A, B, C, D, E, F, a, b, c, d, e, f}
B = {B, C, D, E, F, G, a, b, c, d}
C = {C, D, E, F, G, H, b, c, d, e}
D = {D, E, F, G, H, I, c, d, e, f}
E = {E, F, G, H, I, J, d, e, f}

B ∩ C = {C, D, E, F, G, b, c, d}
A ⊕ (B ∩ C) = {A, B, G, a, e, f}
(A ⊕ (B ∩ C)) \ D = {A, B, a}
((A ⊕ (B ∩ C)) \ D) ∩ E = {}

=== Example 2: Random Sets ===
A = {D, I, J, K, M, R, S, V, Z, c}
B = {D, F, G, H, L, N, O, P, T, U, Y, a, c, d, f}
C = {C, H, I, L, M, N, U, W, Z, e}
D = {A, G, J, K, P, S, U, V, W, Y, d, f}
E = {E, G, Q, S, a, e, f}

B ∩ C = {H, L, N, U}
A ⊕ (B ∩ C) = {D, H, I, J, K, L, M, N, R, S, U, V, Z, c}
(A ⊕ (B ∩ C)) \ D = {D, H, I, L, M, N, R, Z, c}
((A ⊕ (B ∩ C)) \ D) ∩ E = {}

=== Example 3: Intermediate Results in HashTable ===
A = {A, B, C, D, E, F, G, H, I, J}
B = {C, D, E, F, G, H, I, J, K, L}
C = {E, F, G, H, I, J, K, L, M, N}
D = {G, H, I, J, K, L, M, N, O, P}
E = {I, J, K, L, M, N, O, P, Q, R}
B ∩ C = {E, F, G, H, I, J, K, L}
A ⊕ (B ∩ C) = {A, B, C, D, K, L}
(A ⊕ (B ∩ C)) \ D = {A, B, C, D}
result = {}
```

Рис. 1 — Тестовые прогоны

```

=====
                        SEQUENCE OPERATIONS
=====

===== CONCAT =====
A          = [a, b, c, d]
B          = [e, f]
A.concat(B) = [a, b, c, d, e, f]
A          = [10, 20, 30]
B          = [40, 50]
A.concat(B) = [10, 20, 30, 40, 50]

===== ERASE =====
A          = [a, b, c, d, e, f]
A.erase(p1=2, p2=4) = [a, e, f]
A.erase(p1=1, p2=1) = [b, c, d, e, f]
A.erase(p1=1, p2=6) = []
A.erase(4,2): incorrect input

===== EXCL =====
A          = [a, b, c, d, e, f]
A.excl([b,c,d]) = [a, e, f]
A.excl([x,y])   = [a, b, c, d, e, f]
A.excl(A)       = []
A2           = [a, b, c, a, b, c, d]
A2.excl([a,b,c]) = [a, b, c, d]

=====
                        HASHTABLE WITH SEQUENCE TEST
=====

ht[s1]          = [H, e, l, l, o]
ht[s2]          = [W, o, r, l, d]
ht[s1].concat(ht[s2]) = [H, e, l, l, o, W, o, r, l, d]
ht[s1].erase(2,3)   = [H, l, o]
ht[s1].excl([l,l])  = [H, e, o]

```

Рис. 2 — Тестовые прогоны

9 Результаты тестирования

Были проверены два типа операций: над множествами и над последовательностями. В примерах с множествами по формуле $((A \oplus (B \cap C)) \setminus D) \cap E$ все вычисления дали пустое множество — и в ручном примере, и в случайном, и при использовании хеш-таблицы. Затем тестировали последовательности: слияние (concat) работает правильно, удаление (erase) корректно убирает элементы и ловит ошибки, а исключение подпоследовательностей (excl) удаляет только заданные фрагменты. Хеш-таблица с последовательностями тоже работает без ошибок. В итоге все тесты пройдены успешно.

10 Вывод

В ходе выполнения лабораторной работы был разработан и протестирован набор контейнерных классов, моделирующих поведение стандартных структур данных STL. Контейнер BitMaskSet, основанный на битовой маске, позволяет эффективно выполнять теоретико-множественные операции (объединение, пересечение, разность, симметрическую разность) за константное время $O(1)$, что даёт значительный выигрыш в производительности по сравнению с использованием `std::set`. Контейнер Sequence<T>, построенный на основе `std::vector`, демонстрирует линейную сложность операций удаления и поиска подпоследовательности. Хеш-таблица HashTable<K, V>, реализованная методом цепочек, обеспечивает амортизированное константное время вставки, поиска и удаления при условии хорошей хеш-функции, однако в худшем случае (коллизии всех ключей) сложность может ухудшиться до $O(n)$. Для заданной цепочки операций $((A \oplus (B \cap C)) \setminus D) \cap E$ с использованием BitMaskSet все вычисления выполнены за константное время, а результирующее множество оказалось пустым как для ручного, так и для случайно сгенерированного набора данных, что подтверждает корректность реализации и приоритетов операций. Все тесты успешно пройдены, работа может считаться выполненной.

11 Код программы

1) bmask_set.cpp

```
#include "bmask_set.hpp"

int BitMaskSet::setCounter = 0;
```

```

BitMaskSet::BitMaskSet(){ set = 0UL; }
BitMaskSet::BitMaskSet(const unsigned long _mask){ set = _mask; }

BitMaskSet::BitMaskSet(const char* inpSet){
    set = 0UL;
    if(!inpSet) return;
    for(size_t i = 0; i < strlen(inpSet); i++){
        if(inpSet[i] >= 'A' && inpSet[i] <= 'Z')
            set |= 1UL << (inpSet[i] - 'A');
        else if(inpSet[i] >= 'a' && inpSet[i] <= 'f')
            set |= 1UL << (inpSet[i] - 'a' + 26);
    }
}

BitMaskSet::BitMaskSet(const std::string& inpSet){
    set = 0UL;
    for(size_t i = 0; i < inpSet.length(); i++){
        if(inpSet[i] >= 'A' && inpSet[i] <= 'Z')
            set |= 1UL << (inpSet[i] - 'A');
        else if(inpSet[i] >= 'a' && inpSet[i] <= 'f')
            set |= 1UL << (inpSet[i] - 'a' + 26);
    }
}

BitMaskSet::BitMaskSet(const BitMaskSet& other){ set = other.set; }
BitMaskSet::~~BitMaskSet(){}

BitMaskSet& BitMaskSet::operator=(const BitMaskSet& other){
    if(this != &other) set = other.set;
    return *this;
}

BitMaskSet BitMaskSet::operator|(const BitMaskSet& other)const{
    return BitMaskSet(set | other.set);
}

BitMaskSet BitMaskSet::operator&(const BitMaskSet& other)const{
    return BitMaskSet(set & other.set);
}

BitMaskSet BitMaskSet::operator^(const BitMaskSet& other)const{
    return BitMaskSet(set ^ other.set);
}

BitMaskSet BitMaskSet::operator-(const BitMaskSet& other)const{
    return BitMaskSet(set & ~other.set);
}

BitMaskSet BitMaskSet::operator~()const{
    return BitMaskSet(~set & static_cast<unsigned long>((1ULL << UNIVERSUM_SIZE)
- 1ULL));
}

```

```

}

BitMaskSet& BitMaskSet::operator|=(const BitMaskSet& other){ set |= other.set;
return *this; }
BitMaskSet& BitMaskSet::operator&=(const BitMaskSet& other){ set &= other.set;
return *this; }
BitMaskSet& BitMaskSet::operator^=(const BitMaskSet& other){ set ^= other.set;
return *this; }
BitMaskSet& BitMaskSet::operator-=(const BitMaskSet& other){ set &= ~other.set;
return *this; }

bool BitMaskSet::operator==(const BitMaskSet& other)const{ return set ==
other.set; }

BitMaskSet BitMaskSet::variant36(const BitMaskSet& A, const BitMaskSet& B, const
BitMaskSet& C, const BitMaskSet& D,const BitMaskSet& E)
{
    BitMaskSet tmp1 = B & C;
    BitMaskSet tmp2 = A ^ tmp1;
    BitMaskSet tmp3 = tmp2 - D;
    BitMaskSet result = tmp3 & E;
    return result;
}

BitMaskSet BitMaskSet::subtractSets(const BitMaskSet& B, const BitMaskSet& C,
const BitMaskSet& D)const{
    return *this & ~(B | C | D);
}

char* BitMaskSet::toChar()const{
    char* res = new char[UNIVERSUM_SIZE + 1];
    int resSize = 0;
    for(size_t i = 0; i < UNIVERSUM_SIZE; i++){
        if(set & (1UL << i)){
            res[resSize++] = (i < 26) ? ('A' + i) : ('a' + i - 26);
        }
    }
    res[resSize] = '\\0';
    return res;
}

std::string BitMaskSet::toString()const{
    char* c = toChar();
    std::string s(c);
    delete[] c;
    return s;
}

void BitMaskSet::print()const{
    char* s = toChar();

```

```

        std::cout << "{";
        for(size_t i = 0; i < strlen(s); i++){
            if(i > 0) std::cout << ", ";
            std::cout << s[i];
        }
        std::cout << "}";
        delete[] s;
    }

    void BitMaskSet::insert(char ch){
        if(ch >= 'A' && ch <= 'Z')      set |= 1UL << (ch - 'A');
        else if(ch >= 'a' && ch <= 'f') set |= 1UL << (ch - 'a' + 26);
    }

    void BitMaskSet::remove(char ch){
        if(ch >= 'A' && ch <= 'Z')      set &= ~(1UL << (ch - 'A'));
        else if(ch >= 'a' && ch <= 'f') set &= ~(1UL << (ch - 'a' + 26));
    }

    bool BitMaskSet::contains(char ch)const{
        if(ch >= 'A' && ch <= 'Z')      return set & (1UL << (ch - 'A'));
        if(ch >= 'a' && ch <= 'f')      return set & (1UL << (ch - 'a' + 26));
        return false;
    }

    void BitMaskSet::clear(){ set = 0UL; }
    unsigned long BitMaskSet::getMask()const{ return set; }
    void BitMaskSet::setMask(const unsigned long _mask){ set = _mask; }

```

2) bmask_set.hpp

```

#pragma once
#include <iostream>
#include <cstring>
#include <string>

class BitMaskSet{
private:
    unsigned long set; //set in the form of a bitmask
    static const std::size_t UNIVERSUM_SIZE = 32; //size of latin alphabet
    static int setCounter;
public:
    BitMaskSet();
    BitMaskSet(const unsigned long _mask);
    BitMaskSet(const char* inpSet);
    BitMaskSet(const std::string& inpSet);
    BitMaskSet(const BitMaskSet& other);
    ~BitMaskSet();

    BitMaskSet& operator=(const BitMaskSet& other);

```

```

BitMaskSet operator|(const BitMaskSet& other)const;
BitMaskSet operator&(const BitMaskSet& other)const;
BitMaskSet operator^(const BitMaskSet& other)const;
BitMaskSet operator-(const BitMaskSet& other)const;
BitMaskSet operator~()const;

BitMaskSet& operator|=(const BitMaskSet& other);
BitMaskSet& operator&=(const BitMaskSet& other);
BitMaskSet& operator^=(const BitMaskSet& other);
BitMaskSet& operator-=(const BitMaskSet& other);

bool operator==(const BitMaskSet& other)const;

static BitMaskSet variant36(const BitMaskSet& A, const BitMaskSet& B, const
BitMaskSet& C, const BitMaskSet& D, const BitMaskSet& E);

BitMaskSet subtractSets(const BitMaskSet& B, const BitMaskSet& C, const
BitMaskSet& D)const;

char* toChar()const;
std::string toString()const;
void print()const;
void insert(char ch);
void remove(char ch);
bool contains(char ch)const;
void clear();
unsigned long getMask()const;
void setMask(const unsigned long _mask);
};

```

3) generator.cpp

```

#include "generator.hpp"

Generator::Generator(){
    std::random_device rd;
    rng = std::mt19937(rd());
}

Generator::~~Generator(){}

unsigned long Generator::generateSet(size_t minSize, size_t maxSize){
    unsigned long used = 0UL, mask = 1UL;
    unsigned long randChar;
    bool flag=false;
    size_t setSize = randomInt(minSize, maxSize);
    if(setSize>MAX_SET_SIZE*2/3){
        for(size_t currSetSize=0; currSetSize!=setSize;currSetSize++){
            flag=false;

```

```

        mask = 1UL;
        randChar = randomInt(0, MAX_SET_SIZE-1);
        mask = mask<<randChar;
        if(!(mask & used)){
            used = used|mask;
        }
        else{
            for(int i=randChar+1;i<MAX_SET_SIZE && !flag;i++){
                mask = 1UL;
                mask = mask<<i;
                if(!(mask & used)){
                    used = used|mask;
                    flag=true;
                }
            }
            for(int i=randChar-1;i>-1 && !flag;i--){
                mask = 1UL;
                mask = mask<<i;
                if(!(mask & used)){
                    used = used|mask;
                    flag=true;
                }
            }
        }
    }
}
else{
    for(size_t currSetSize=0; currSetSize!=setSize;){
        mask = 1UL;
        randChar = randomInt(0, MAX_SET_SIZE-1);
        mask = mask<<randChar;
        if(!(mask & used)){
            used = used|mask;
            currSetSize++;
        }
    }
}
return used;
}

int Generator::randomInt(int min, int max){
    return (rng()%(max+1-min))+min;
}

```

4) generator.hpp

```

#pragma once
#include <iostream>
#include <random>
#define MAX_SET_SIZE 32

```

```

class Generator{
private:
    std::mt19937 rng;
    int randomInt(int min, int max); //generates num in [min, max] (including min
and max)
public:
    Generator();
    ~Generator();
    unsigned long generateSet(size_t minSize, size_t maxSize);
};

```

5) HashTable.hpp

```

#pragma once
#include <vector>
#include <list>
#include <functional>
#include <iterator>
#include <stdexcept>

#include "bmask_set.hpp"

template<typename TK, typename TV>
class HashTable
{
private:
    size_t numBuckets;
    size_t numElems;
    std::vector<std::list<std::pair<TK, TV>>> buckets;
    std::hash<TK> hasher;
    unsigned getBucketIndex(const TK& key);
public:
    HashTable(const size_t _numBuckets=43);
    HashTable(const HashTable& other);
    ~HashTable();

    HashTable& operator=(const HashTable& other);
    TV& operator[](const TK& key);
    void insert(const TK& key, const TV& value);
    TV& getElem(const TK& key);
    bool contains(const TK& key);
    void erase(const TK& key);
    void printAllValues();

    size_t getNumElems();
    size_t getNumBuckets();

    class Iterator
    {
    public:

```

```

using iterator_category = std::forward_iterator_tag;
using value_type         = std::pair<const TK, TV>;
using difference_type    = std::ptrdiff_t;
using pointer            = value_type*;
using reference           = value_type&;

private:
    using BucketVec = std::vector<std::list<std::pair<TK, TV>>>;
    using BucketIter = typename BucketVec::iterator;
    using ElemIter   = typename std::list<std::pair<TK, TV>>::iterator;

    BucketVec* buckets;
    BucketIter bucketIt;
    ElemIter   elemIt;

    void skipEmpty() {
        while (bucketIt != buckets->end() && bucketIt->empty()) {
            ++bucketIt;
        }
        if (bucketIt != buckets->end()) {
            elemIt = bucketIt->begin();
        }
    }

public:
    Iterator(BucketVec* bkts, BucketIter bIt)
        : buckets(bkts), bucketIt(bIt)
    {
        skipEmpty();
    }

    Iterator(BucketVec* bkts, BucketIter bIt, ElemIter eIt)
        : buckets(bkts), bucketIt(bIt), elemIt(eIt) {}

    reference operator*() {
        return reinterpret_cast<reference>(*elemIt);
    }

    pointer operator->() {
        return reinterpret_cast<pointer>(&(*elemIt));
    }

    Iterator& operator++() {
        ++elemIt;
        if (elemIt == bucketIt->end()) {
            ++bucketIt;
            skipEmpty();
        }
        return *this;
    }

```

```

    Iterator operator++(int) {
        Iterator tmp = *this;
        ++(*this);
        return tmp;
    }

    bool operator==(const Iterator& other) const {
        if (bucketIt != other.bucketIt) return false;
        if (bucketIt == buckets->end()) return true;
        return elemIt == other.elemIt;
    }

    bool operator!=(const Iterator& other) const {
        return !(*this == other);
    }
};

Iterator begin(){
    return Iterator(&buckets, buckets.begin());
}

Iterator end(){
    return Iterator(&buckets, buckets.end(), {});
}
};

template<typename TK, typename TV>
HashTable<TK, TV>::~HashTable(){}

template<typename TK, typename TV>
HashTable<TK, TV>::HashTable(const size_t _numBuckets):numBuckets(_numBuckets){
    buckets.resize(numBuckets);
    numElems=0;
}

template<typename TK, typename TV>
HashTable<TK, TV>::HashTable(const HashTable& other){
    numBuckets=other.numBuckets;
    buckets.resize(numBuckets);
    numElems=other.numElems;
    auto iter = buckets.begin();
    for(const auto& bucket:other.buckets){
        *iter = bucket;
        iter++;
    }
}

template<typename TK, typename TV>
TV& HashTable<TK, TV>::operator[](const TK& key){
    unsigned bucketIndex = getBucketIndex(key);

```

```

        for(auto& elem:buckets[bucketIndex]){
            if(elem.first == key){
                return elem.second;
            }
        }

        buckets[bucketIndex].push_back(std::make_pair(key, TV()));
        numElems++;

        return buckets[bucketIndex].back().second;
    }

template<typename TK, typename TV>
HashTable<TK, TV>& HashTable<TK, TV>::operator=(const HashTable& other){
    if(this==&other){
        return *this;
    }

    numBuckets = other.numBuckets;
    numElems = other.numElems;
    hasher = other.hasher;

    buckets.clear();
    buckets.resize(numBuckets);

    for (size_t i=0; i<numBuckets; i++) {
        buckets[i] = other.buckets[i];
    }

    return *this;
}

template<typename TK, typename TV>
unsigned HashTable<TK, TV>::getBucketIndex(const TK& key){
    return hasher(key)%numBuckets;
}

template<typename TK, typename TV>
void HashTable<TK, TV>::insert(const TK& key, const TV& value){
    unsigned bucketIndex = getBucketIndex(key);

    for(auto& elem:buckets[bucketIndex]){
        if(elem.first==key){
            elem.second=value;
            return;
        }
    }

    buckets[bucketIndex].push_back(std::make_pair(key, value));
    numElems++;
}

```

```

}

template<typename TK, typename TV>
TV& HashTable<TK, TV>::getElem(const TK& key){
    unsigned idx = getBucketIndex(key);
    for(auto& elem:buckets[idx]){
        if(elem.first==key){
            return elem.second;
        }
    }
    throw std::invalid_argument("NO SUCH KEY!\n"); // never reached
}

template<typename TK, typename TV>
bool HashTable<TK, TV>::contains(const TK& key){
    unsigned bcktIndex = getBucketIndex(key);
    for(const auto& elem:buckets[bcktIndex]){
        if(elem.first==key){
            return true;
        }
    }
    return false;
}

template<typename TK, typename TV>
void HashTable<TK, TV>::erase(const TK& key){
    if(!contains(key)){
        throw std::invalid_argument("NO SUCH KEY!\n");
    }

    unsigned bcktIndex = getBucketIndex(key);
    auto iter = buckets[bcktIndex].begin();
    auto endIter = buckets[bcktIndex].end();
    while(iter!=endIter){
        if((*iter).first==key){
            buckets[bcktIndex].erase(iter);
            numElems--;
            return;
        }
        iter++;
    }
}

template<typename TK, typename TV>
void HashTable<TK, TV>::printAllValues(){
    int i=0;
    for(const auto& bucket:buckets){
        if(!bucket.empty()){
            std::cout << "BUCKET INDEX: " << i << '\n';
            for(const auto& elem:bucket){

```

```

        std::cout << "Key: " << elem.first << ", Value: " << elem.second
<< "; ";
    }
    std::cout << '\n';
    std::cout << '\n';
}
i++;
}
}

template<typename TK, typename TV>
size_t HashTable<TK, TV>::getNumBuckets(){return numBuckets;}

template<typename TK, typename TV>
size_t HashTable<TK, TV>::getNumElems(){return numElems;}

```

6) Interface.hpp

```

#pragma once
#include "HashTable.hpp"
#include "sequence.hpp"
#include "generator.hpp"
#include <iostream>
#include <string>

class Interface {
public:
    Interface();
    ~Interface();

    void show();
};

```

7) Interface.cpp

```

// Consolidated Interface.cpp - Combined output of both interfaces
#include "Interface.hpp"
#include <iostream>
#include <string>

Interface::Interface() {}
Interface::~Interface() {}

static void printSet(const std::string& name, const BitMaskSet& s) {
    std::cout << name << " = ";
    s.print();
    std::cout << "\n";
}

void Interface::show() {

```

```

std::cout <<
"=====\n";
std::cout << "  Variant 36 | Power: 32 | (A  $\oplus$  B  $\cap$  C)  $\setminus$  D  $\cap$  E\n";
std::cout << "  Priority:  $\cap$  >  $\oplus$  >  $\setminus$  >  $\cup$ \n";
std::cout << "  Formula expands as: ((A  $\oplus$  (B  $\cap$  C))  $\setminus$  D)  $\cap$  E\n";
std::cout <<
"=====\n\n";

Generator gen;

std::cout << "=== Example 1: Manual Sets ===\n";
{
    BitMaskSet A("ABCDEFabcdef");
    BitMaskSet B("BCDEFGabcd");
    BitMaskSet C("CDEFGHbcde");
    BitMaskSet D("DEFGHIcdef");
    BitMaskSet E("EFGHIJdef");

    printSet("A", A);
    printSet("B", B);
    printSet("C", C);
    printSet("D", D);
    printSet("E", E);
    std::cout << "\n";

    BitMaskSet step1 = B & C;
    printSet("B  $\cap$  C", step1);

    BitMaskSet step2 = A ^ step1;
    printSet("A  $\oplus$  (B  $\cap$  C)", step2);

    BitMaskSet step3 = step2 - D;
    printSet("(A  $\oplus$  (B  $\cap$  C))  $\setminus$  D", step3);

    BitMaskSet result = step3 & E;
    printSet("((A  $\oplus$  (B  $\cap$  C))  $\setminus$  D)  $\cap$  E", result);
}

std::cout << "\n=== Example 2: Random Sets ===\n";
{
    BitMaskSet A(gen.generateSet(5, 20));
    BitMaskSet B(gen.generateSet(5, 20));
    BitMaskSet C(gen.generateSet(5, 20));
    BitMaskSet D(gen.generateSet(5, 20));
    BitMaskSet E(gen.generateSet(5, 20));

    printSet("A", A);
    printSet("B", B);
    printSet("C", C);
    printSet("D", D);
    printSet("E", E);
}

```

```

std::cout << "\n";

BitMaskSet step1 = B & C;
printSet("B n C", step1);

BitMaskSet step2 = A ^ step1;
printSet("A  $\oplus$  (B n C)", step2);

BitMaskSet step3 = step2 - D;
printSet("(A  $\oplus$  (B n C)) \\ D", step3);

BitMaskSet result = step3 & E;
printSet("((A  $\oplus$  (B n C)) \\ D) n E", result);
}

std::cout << "\n=== Example 3: Intermediate Results in HashTable ===\n";
{
    BitMaskSet A("ABCDEFGH IJ");
    BitMaskSet B("CDEFGH IJ KL");
    BitMaskSet C("EFGH IJ KL MN");
    BitMaskSet D("GH IJ KL MN OP");
    BitMaskSet E("I J KL MN OP Q R");

    HashTable<std::string, BitMaskSet> steps;
    steps.insert("A", A);
    steps.insert("B", B);
    steps.insert("C", C);
    steps.insert("D", D);
    steps.insert("E", E);
    steps.insert("BnC", steps.getElem("B") & steps.getElem("C"));
    steps.insert("A $\oplus$ (BnC)", steps.getElem("A") ^ steps.getElem("BnC"));
    steps.insert("(A $\oplus$ BnC)\\D", steps.getElem("A $\oplus$ (BnC)") -
steps.getElem("D"));
    steps.insert("result", steps.getElem("(A $\oplus$ BnC)\\D") &
steps.getElem("E"));

    for (const std::string& key : {"A", "B", "C", "D", "E", "BnC", "A $\oplus$ (BnC)",
"(A $\oplus$ BnC)\\D", "result"}) {
        std::cout << key << " = ";
        steps.getElem(key).print();
        std::cout << "\n";
    }
}

std::cout << "\n\n===== \n";
std::cout << "          SEQUENCE OPERATIONS\n";
std::cout << "===== \n\n";

std::cout << "===== CONCAT ===== \n";
{
    Sequence<char> A = {'a', 'b', 'c', 'd'};

```

```

    Sequence<char> B = {'e', 'f'};
    std::cout << "A          = "; A.print(); std::cout << "\n";
    std::cout << "B          = "; B.print(); std::cout << "\n";
    std::cout << "A.concat(B) = "; A.concat(B).print(); std::cout << "\n";
}
{
    Sequence<int> A = {10, 20, 30};
    Sequence<int> B = {40, 50};
    std::cout << "A          = "; A.print(); std::cout << "\n";
    std::cout << "B          = "; B.print(); std::cout << "\n";
    std::cout << "A.concat(B) = "; A.concat(B).print(); std::cout << "\n";
}
std::cout << "\n";

std::cout << "===== ERASE =====\n";
{
    Sequence<char> A = {'a', 'b', 'c', 'd', 'e', 'f'};
    std::cout << "A          = "; A.print(); std::cout << "\n";

    std::cout << "A.erase(p1=2, p2=4) = "; A.erase(2, 4).print(); std::cout
<< "\n";
    std::cout << "A.erase(p1=1, p2=1) = "; A.erase(1, 1).print(); std::cout
<< "\n";
    std::cout << "A.erase(p1=1, p2=6) = "; A.erase(1, 6).print(); std::cout
<< "\n";

    try {
        A.erase(4, 2);
    } catch (const std::out_of_range& e) {
        std::cout << "A.erase(4,2): incorrect input\n";
    }
}
std::cout << "\n";

std::cout << "===== EXCL =====\n";
{
    Sequence<char> A = {'a', 'b', 'c', 'd', 'e', 'f'};
    std::cout << "A          = "; A.print(); std::cout << "\n";

    Sequence<char> sub1 = {'b', 'c', 'd'};
    std::cout << "A.excl([b,c,d]) = "; A.excl(sub1).print(); std::cout <<
"\n";

    Sequence<char> sub2 = {'x', 'y'};
    std::cout << "A.excl([x,y])   = "; A.excl(sub2).print(); std::cout <<
"\n";

    Sequence<char> sub3 = {'a', 'b', 'c', 'd', 'e', 'f'};
    std::cout << "A.excl(A)       = "; A.excl(sub3).print(); std::cout <<
"\n";
}

```

```

Sequence<char> A2 = {'a', 'b', 'c', 'a', 'b', 'c', 'd'};
Sequence<char> sub4 = {'a', 'b', 'c'};
std::cout << "A2" = "; A2.print(); std::cout << "\n";
std::cout << "A2.excl([a,b,c]) = "; A2.excl(sub4).print(); std::cout <<
"\n";
}
std::cout << "\n";

std::cout << "=====\n";
std::cout << "    HASHTABLE WITH SEQUENCE TEST\n";
std::cout << "=====\n\n";
{
    HashTable<std::string, Sequence<char>> ht;
    ht["s1"] = Sequence<char>({'H', 'e', 'l', 'l', 'o'});
    ht["s2"] = Sequence<char>({'W', 'o', 'r', 'l', 'd'});
    std::cout << "ht[s1]" = "; ht.getElem("s1").print();
std::cout << "\n";
    std::cout << "ht[s2]" = "; ht.getElem("s2").print();
std::cout << "\n";

    Sequence<char> merged = ht.getElem("s1").concat(ht.getElem("s2"));
    std::cout << "ht[s1].concat(ht[s2]) = "; merged.print(); std::cout <<
"\n";

    Sequence<char> trimmed = ht.getElem("s1").erase(2, 3);
    std::cout << "ht[s1].erase(2,3) = "; trimmed.print(); std::cout <<
"\n";

    Sequence<char> sub = {'l', 'l'};
    std::cout << "ht[s1].excl([1,1]) = ";
ht.getElem("s1").excl(sub).print(); std::cout << "\n";
}
}

```

8) sequence.hpp

```

#pragma once
#include <iostream>
#include <vector>
#include <stdexcept>

template<typename T>
class Sequence {
private:
    std::vector<T> data;

public:
    Sequence() = default;

    Sequence(std::initializer_list<T> lst) : data(lst) {}

```

```

explicit Sequence(const std::vector<T>& v) : data(v) {}

Sequence(const Sequence& other) = default;
Sequence& operator=(const Sequence& other) = default;
~Sequence() = default;

T& at(size_t pos) {
    if (pos < 1 || pos > data.size())
        throw std::out_of_range("Sequence::at – индекс вне диапазона");
    return data[pos - 1];
}

const T& at(size_t pos) const {
    if (pos < 1 || pos > data.size())
        throw std::out_of_range("Sequence::at – индекс вне диапазона");
    return data[pos - 1];
}

size_t size() const { return data.size(); }
bool empty() const { return data.empty(); }
void push_back(const T& val) { data.push_back(val); }
void clear() { data.clear(); }

bool operator==(const Sequence& other) const { return data == other.data; }
bool operator!=(const Sequence& other) const { return !(*this == other); }

Sequence concat(const Sequence& other) const {
    Sequence result(*this);
    for (const T& val : other.data)
        result.data.push_back(val);
    return result;
}

Sequence erase(size_t p1, size_t p2) const {
    if (p1 < 1 || p2 > data.size() || p1 > p2)
        throw std::out_of_range(
            "Sequence::erase – некорректные границы [p1, p2]");

    Sequence result;
    for (size_t i = 0; i < data.size(); ++i) {
        size_t pos = i + 1;
        if (pos < p1 || pos > p2)
            result.data.push_back(data[i]);
    }
    return result;
}

Sequence excl(const Sequence& sub) const {
    if (sub.empty() || sub.size() > data.size())
        return *this;
}

```

```

    size_t n = data.size();
    size_t m = sub.data.size();

    for (size_t i = 0; i + m <= n; ++i) {
        bool match = true;
        for (size_t j = 0; j < m && match; ++j)
            if (!(data[i + j] == sub.data[j])) match = false;

        if (match)
            return erase(i + 1, i + m);
    }

    return *this;
}

void print() const {
    std::cout << "[";
    for (size_t i = 0; i < data.size(); ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << data[i];
    }
    std::cout << "]";
}

};

```